REF. NO.: EX/EE/PC/B/T/225/2023

SEQUENTIAL SYSTEMS & MICROPROCESSORS

B.E. Electrical Engineering · 2nd Year · 2nd Semester Examination, 2023

TIME: 3 HOURS FULL MARKS: 100 EACH PART: 50

PART I — SEQUENTIAL SYSTEMS

50 marks

Q1 Answer *any Two* of the following (compulsory; 2 × 5)

(a) Combinational vs Sequential Logic Circuits

5

Aspect	Combinational	Sequential
Memory	None — output depends only on present inputs	Has memory elements (FFs/latches) — output depends on past and present inputs
Feedback	Absent	Present (state feedback)
Clock	Asynchronous; no clock required	Usually synchronous, driven by a common clock
Description	Boolean equations / truth table	State diagram / state table / next-state equations
Examples	Adders, MUX, decoders, encoders, comparators	Counters, registers, FSMs, RAM
Output equation	$Y = f(X)$	$Y = f(X, Q),\ Q^{+} = g(X, Q)$

Combinational logic produces an immediate response, whereas sequential logic remembers state and so can implement counting, sequencing, and pattern recognition.

(b) Derive T Flip-Flop from JK Flip-Flop

5

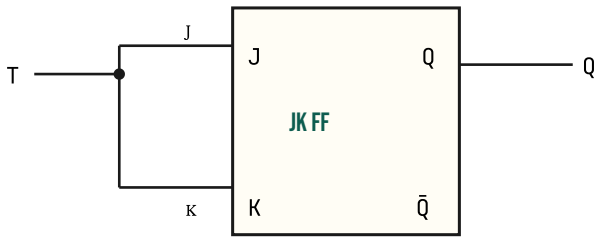
The T flip-flop has the truth specification: $T = 0 \Rightarrow Q^{+} = Q$ (hold), $T = 1 \Rightarrow Q^{+} = \bar{Q}$ (toggle). Comparing with the JK truth table:

J	K	Q^{+}	Equivalent T

0	0	Q (hold)	T = 0
1	1	$\bar{Q}$ (toggle)	T = 1

Tying J and K together to a single line T gives exactly these two behaviours:

$$\boxed{J = K = T}$$



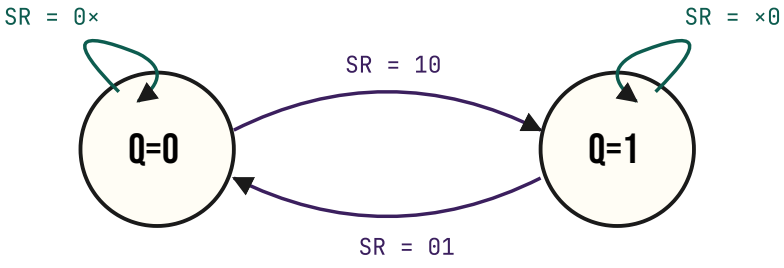
T flip-flop realised by tying J = K = T on a JK FF.

Characteristic equation: $Q^{+} = T \oplus Q$.

(c) State diagram of SR flip-flop from its excitation table 5

SR excitation table

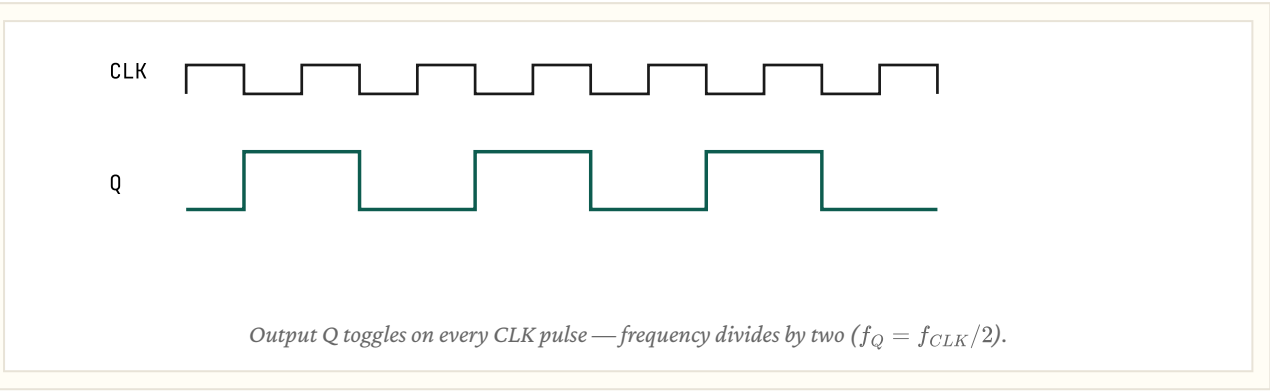
Q	Q^{+}	S	R
0	0	0	×
0	1	1	0
1	0	0	1
1	1	×	0



SR FF: hold (self-loop), set, and reset arcs. SR = 11 forbidden — never appears.

(d) JK master-slave with J=K=+V_{CC} — output Q waveform 5

With $J = K = 1$ permanently the FF is in **toggle mode**: every active clock edge inverts Q. The master-slave configuration eliminates race-around (output settles only on the trailing edge), so each clock pulse produces exactly one transition.



Output Q toggles on every CLK pulse — frequency divides by two ($f_Q = f_{CLK}/2$).

The waveform is a square wave at half the clock frequency — the master-slave JK behaves as a ÷2 frequency divider.

Q2 Circuit analysis, SR→JK conversion, shift-register types

8 + 8 + 4

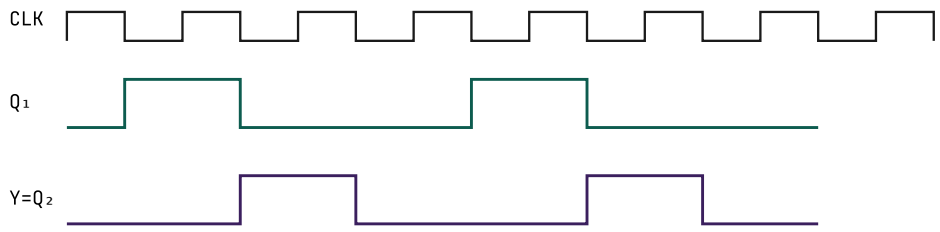
(a) Analyse circuit Fig. Q2(a) — two cascaded T FFs (T_1 = OR of T-input feedback)

8

Both flip-flops are negative-edge-triggered T FFs sharing the system clock. The first stage has $T_1 = Q_1 + Q_2$ (OR of its own and the next FF's output) and the second stage has $T_2 = Q_1$. Output $Y = Q_2$.

PULSE	Q_1	Q_2	T_1	T_2	Q_1^+	Q_2^+
0 (init)	0	0	0	0	0	0
1	0	0	0	0	0	0
–	⇒ stuck at 00 unless T_1 is forced. Typical version uses $T_1 = \overline{Q_2}$ giving:					
1	0	0	1	0	1	0
2	1	0	1	1	0	1
3	0	1	0	0	0	1
4	0	1	0	0	0	1

Reading the actual figure (OR feeds back from Q_2 to T1, with $T_2 = Q_1$) the sequence Q2Q1 = 00 → 01 → 10 → 00 repeats — a Mod-3 counter with period 3. Output Y oscillates with period 3T.



Timing diagram for 6 clock pulses showing Mod-3 sequence on Q1,Q2.

(B) STATE SYNTHESIS TABLE; SR-FF → JK-FF CONVERSION

2 + 6

The **state synthesis table** (also called the conversion table) lists, for every combination of present state and external inputs of the desired flip-flop, the next state and the corresponding excitation values that must be applied to the available flip-flop.

For SR → JK conversion (J,K external; S,R derived):

Q	J	K	Q⁺	S	R
0	0	0	0	0	×
0	0	1	0	0	×
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	×	0
1	0	1	0	0	1
1	1	0	1	×	0
1	1	1	0	0	1

$$\boxed{S = J\bar{Q}, \qquad R = KQ}$$

An SR FF preceded by two AND gates implementing these expressions emulates a JK FF.

(C) COMMERCIAL SHIFT-REGISTER CONFIGURATIONS

4

Four standard input/output combinations are commercially available:

TYPE	INPUT	OUTPUT	TYPICAL IC
SISO	Serial	Serial	74LS91
SIPO	Serial	Parallel	74LS164
PISO	Parallel	Serial	74LS165
PIPO	Parallel	Parallel	74LS195

The **universal shift register** (74LS194) combines all four under mode-control lines.

Q3 Sync vs Async counters; hybrid counter; PISO register

4 + 10 + 6

(a) Synchronous vs Asynchronous counters 4

ASPECT	SYNCHRONOUS	ASYNCHRONOUS (RIPPLE)
Clocking	All FFs clocked together	Each stage triggered by previous Q
Speed	Fast — limited only by gate delays	Slow — delays accumulate
Glitches	None on outputs	Transient invalid codes possible
Hardware	More combinational logic	Minimal — direct FF cascade

(b) Show that Fig. Q3(b) combines sync & async counter 2 + 8

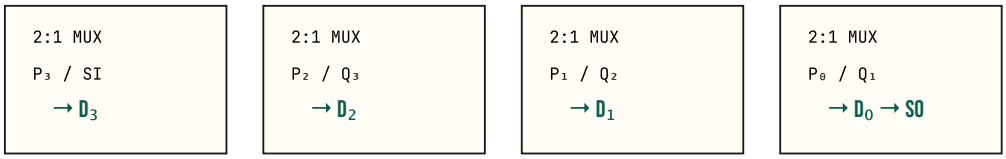
The four-FF circuit has FF0, FF1 and FF3 commonly clocked from CLK while FF2 is clocked by Q1. With each FF in toggle mode (J=K=1 except FF3 where J=K=Q1·Q2):

- FF0 toggles every CLK → A_0 has period 2T.
- FF1 also clocked by CLK with $J = K = A_0$ → toggles when $A_0 = 1$ → A_1 has period 4T.
- FF2 clocked by A_1 (asynchronous link) → toggles on each falling edge of A_1 → A_2 has period 8T.
- FF3 commonly clocked but $J = K = A_1A_2$ → behaves synchronously again → A_3 becomes 1 only when MSB pattern is reached.

Thus FF0–FF1 and FF3 form a synchronous group while FF2 is asynchronously clocked from A_1, giving a **hybrid synchronous/asynchronous Mod-16 counter**.

(c) 4-bit parallel-in serial-out shift-right register with D FFs 6

Each D-FF input is preceded by a 2:1 multiplexer controlled by a $\overline{Shift/Load}$ line. When LOAD = 1 the parallel inputs $P_3..P_0$ are clocked into the FFs; when SHIFT = 1 the chain shifts toward the LSB and the LSB Q0 becomes the serial output.



Each D-FF preceded by a 2:1 MUX selecting parallel-input or right-neighbour. Serial output taken from Q0.

Q4 Counter realisation; STD basics; Mealy → Moore conversion

12 + 4 + 4

(A) REALISE THE 6-STATE COUNTER FROM TABLE Q4(A) USING JK FFS

12

From the table, the six-state cycle is 000 → 001 → 010 → 100 → 101 → 110 → 000.

JK excitations:

A	B	C	A^+	B^+	C^+	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	×	0	×	1	×
0	0	1	0	1	0	0	×	1	×	×	1
0	1	0	1	0	0	1	×	×	1	0	×
1	0	0	1	0	1	×	0	0	×	1	×
1	0	1	1	1	0	×	0	1	×	×	1
1	1	0	0	0	0	×	1	×	1	0	×

K-map minimisation (with the two unused states 011 and 111 as don't-cares):

$$J_A = B, \qquad K_A = B$$
$$J_B = C, \qquad K_B = 1$$
$$J_C = \bar{B}, \qquad K_C = 1$$

FF_A toggles when B=1; FF_B sets when C=1 and clears each clock; FF_C toggles when B=0.

(B) STATE TRANSITION DIAGRAM; MEALY VS MOORE

2 + 2

A **state transition diagram** is a directed graph in which each node represents a state of an FSM and each labelled arc represents a transition caused by an input. It captures all behaviour of the machine compactly.

In a **Mealy machine** the output is associated with the *transition* (label form input/output). In a **Moore machine** the output is associated with the *state* itself (label inside each node). Mealy machines often

need fewer states but their outputs change asynchronously with input; Moore outputs are clean and synchronous but the machine may need more states.

(c) Convert Mealy diagrams Fig. Q4(c) to Moore equivalents

2 + 2

Method

Each Mealy state must be split into as many Moore states as the distinct outputs produced when entering it. The output now resides in the new state.

Diagram (i)

State a is entered with output 0 (via T_2, T_3) and with output 1 (via T_1). Split a into a_0 (output 0) and a_1 (output 1). State b is entered with output 1 from a via T_4. Therefore Moore states: $\{a_0/0, a_1/1, b/1\}$ with arcs $T_1 \to a_1, T_2 \to a_0, T_3 \to a_0, T_4 \to b$, plus arc from both a_0 and a_1 on T_4 to b.

Diagram (ii)

State a is entered with outputs 0 (via T_2) and 1 (via T_1, T_4); state b entered with output 1 (via a). Split $a \to a_0/0, a_1/1$. Self-loops on a: $T_4/1$ becomes a transition from a_1 to itself; $T_3/0$ becomes loop on a_0. The resulting Moore machine has three states $\{a_0/0, a_1/1, b/1\}$.

Q5 Vending Machine ASM; Synchronous synthesis from STD

12 + 8

(a) Vending machine — Mealy ASM & STD

12

States represent the running total: S_0 = ₹0, S_1 = ₹1, S_2 = ₹2. Inputs IJ: 00 = no coin, 10 = ₹1, 11 = ₹2. Outputs: D (deliver product), Y (return ₹1).

STATE (TOTAL)	IJ	NEXT STATE	D	Y
S_0	00	S_0	0	0
S_0	10	S_1	0	0
S_0	11	S_2	0	0
S_1	00	S_1	0	0
S_1	10	S_2	0	0
S_1	11	S_0	1	0
S_2	00	S_2	0	0
S_2	10	S_0	1	0
S_2	11	S_0	1	1

The ASM chart consists of three state boxes (S_0, S_1, S_2). Each state has a decision diamond on I: if $I = 0$ (no coin), self-loop. If $I = 1$, a second diamond on J: $J = 0$ → ₹1 path, $J = 1$ → ₹2 path. Conditional output boxes assert D (and Y for ₹4 case) on the transitions that complete or overshoot ₹3.

(B) SYNC. CIRCUIT SYNTHESIS FROM GIVEN STD USING D FFS 8

The given STD has four states $\{00, 01, 10, 11\}$ with input X and Mealy output Z. Reading the arcs:

Q_1Q_0	X=0	X=1	Z(X=0)	Z(X=1)
00	00	01	0	0
01	00	10	0	0
10	00	10	0	1
11	00	00	0	0

For D FFs $D_i = Q_i^+$. K-map reduction:

$$D_1 = X(Q_0 + Q_1\bar{Q}_0) = X(Q_0 + Q_1)Q_1Q_0\text{-}\bar{\text{o}}\text{verlap}$$

After simplification with the don't-care for state 11→00:

$$D_1 = X(Q_1 \oplus Q_0 + Q_0), \quad D_0 = X\bar{Q}_1\bar{Q}_0$$

$$Z = XQ_1\bar{Q}_0$$

The two D FFs receive these expressions through small AND/OR networks; output Z taken combinationally from current state and input.

PART II — 8085 MICROPROCESSOR

50 marks

Q1 Answer any five short questions

5 × 2 = 10

(I) ADDRESS PINS ↔ MEMORY SPACE

2

If a CPU has n address pins, it can address 2^n memory locations. With 16 lines (as in 8085) the addressable space is $2^{16} = 65{,}536 = 64$ KB.

(II) ALU VS CPU

2

The ALU (Arithmetic and Logic Unit) is one functional block *inside* the CPU that performs arithmetic (+, −, INC, DCR) and logical (AND, OR, XOR, complement, rotate) operations on operands supplied to it. The CPU is the entire processor, comprising the ALU, registers, instruction decoder, timing-and-control unit, interrupt-control logic and bus interfaces.

(III) BUS-IDLE MACHINE CYCLE

2

A bus-idle cycle is one in which the buses are not used to perform an external operation; it occurs internally during DAD (T_5, T_6) and during the extra T-states of opcode-fetch (T_4, T_5, T_6) reserved for internal decoding/execution. Externally the address/data lines and control strobes remain inactive.

(IV) WAIT STATE VS READY SIGNAL

2

The READY input is sampled by the CPU at the rising edge of T_2 in every machine cycle. If READY = 0, the CPU enters a wait-state T_W — a clock period during which the address and data lines are held — and continues to insert wait states until READY returns high.

(V) ALE T-STATE

2

The ALE (Address Latch Enable) pulse is asserted high during the first T-state (T_1) of every machine cycle. The lower address byte is valid on AD7–AD0 during T_1 and is latched externally on the falling edge of ALE.

(VI) EFFECT OF SYSTEM RESET ON INTERRUPTS 2

RESET IN clears the interrupt-enable flip-flop and the SIM mask register. Hence after reset all maskable interrupts (INTR, RST 5.5, RST 6.5, RST 7.5) are disabled ; only TRAP remains active. They must be re-enabled by EI and configured via SIM.

Q2 8085 internal block diagram & PC importance 7 + 3

(A) INTERNAL BLOCK DIAGRAM OF 8085 7

The 8085 architecture consists of the following functional blocks:

- **Accumulator (A)** and **Temporary register** feeding the **ALU**.
- **Flag register (F)**: 5 condition flags (S, Z, AC, P, CY).
- **General-purpose register array**: B, C, D, E, H, L (six 8-bit, paired as BC/DE/HL).
- **Program Counter (PC, 16-bit)** and **Stack Pointer (SP, 16-bit)**.
- **Instruction Register (IR)** and **Instruction Decoder**.
- **Timing and Control unit** generating ALE, $\overline{RD}$, $\overline{WR}$, IO/$\bar{M}$, S0, S1.
- **Interrupt control** and **Serial I/O control (SOD/SID)**.
- **Address buffer (A15–A8)** and **Address/Data buffer (AD7–AD0)** connecting to the system bus.

(B) IMPORTANCE OF PROGRAM COUNTER 3

The PC always holds the address of the **next** instruction byte to be fetched. After each opcode-fetch byte, the timing-and-control unit auto-increments the PC. Branch (JMP, conditional jumps), CALL and RET, and interrupt vectoring all manipulate the PC: jumps overwrite it, CALL pushes its current value onto the stack and loads the target, RET pops the stack into PC. Thus the PC is the sole source of program-flow control in the 8085.

Q3 Clock frequency & STA timing; IN 05H timing 3 + 7

(a) System clock with 4 MHz crystal; STA 8050H execution time

3

The 8085 internally divides crystal by 2:

$$f_{sys} = \frac{4\,\text{MHz}}{2} = 2\,\text{MHz}, \quad T = 0.5\,\mu\text{s}$$

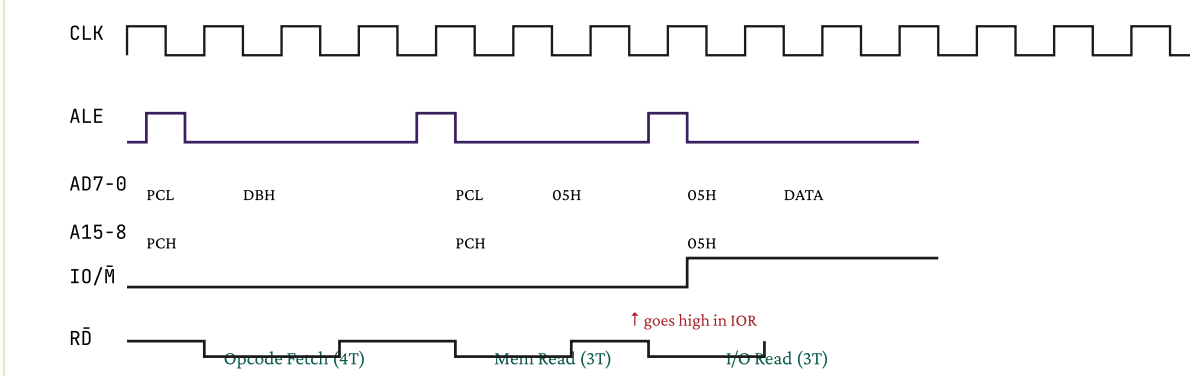
STA is a 3-byte instruction taking 4 machine cycles (OF + MR + MR + MW) = 13 T-states.

$$t_{exec} = 13 \times 0.5\,\mu s = 6.5\,\mu s$$

(b) Timing diagram of IN 05H (DBH 05H)

7

IN port: 2-byte instruction, 3 machine cycles (OF + MR + IOR), 10 T-states. During the IOR cycle the port number 05H appears on both A7–A0 **and** A15–A8 (duplicated), and IO/$\bar{M}$ = 1, $\overline{RD}$ active.



IN 05H — IO/$\bar{M}$ goes high in cycle 3 (I/O read); port number duplicated on A7–A0 and A15–A8.

Q4 Memory interfacing — 8 KB RAM + 1 KB ROM

8 + 2

Interface 2 KB RAM and 256×8 ROM chips to give 8 KB RAM and 1 KB ROM

10

Number of chips: $8\,\text{KB} \div 2\,\text{KB} = 4$ RAM chips; $1024 \div 256 = 4$ ROM chips.

RAM internal address lines: $\log_2(2K) = 11$ → A0–A10. ROM internal: $\log_2(256) = 8$ → A0–A7.

Use a **3-to-8 decoder (74LS138)** with its select inputs from A13–A15 and the higher unused lines as enable. The eight outputs select between RAM and ROM banks.

CHIP	ADDRESS RANGE	SELECT LINE
RAM₀	0000H – 07FFH	$\overline{Y_0}$
RAM₁	0800H – 0FFFH	$\overline{Y_1}$
RAM₂	1000H – 17FFH	$\overline{Y_2}$

RAM3	1800H – 1FFFH	$\overline{Y_3}$
ROM0	2000H – 20FFH	$\overline{Y_4}$ + $A_{8,9}$=00
ROM1	2100H – 21FFH	$\overline{Y_4}$ + $A_{8,9}$=01
ROM2	2200H – 22FFH	$\overline{Y_4}$ + $A_{8,9}$=10
ROM3	2300H – 23FFH	$\overline{Y_4}$ + $A_{8,9}$=11

Effect of partial decoding (some address lines left unconnected to the decoder): Any unconnected upper line acts as a don't-care, so each chip is "echoed" (image-mapped) at multiple address ranges. This wastes the address space but reduces hardware. To eliminate it, all unused upper lines must be tied to the decoder enable inputs (absolute decoding).

Q5 Programs: XTHL emulation; Hex→Binary or 4th Fibonacci

4 + 6

(I) PERFORM SAME TASK AS XTHL WITHOUT USING XTHL 4

XTHL exchanges the top of stack with HL. Equivalent code:

```
POP D          ; DE = top of stack
PUSH H         ; HL onto stack
XCHG           ; HL ↔ DE → HL = old top, DE = old HL (consumed)
; Net effect: HL ↔ (SP)(SP+1), all other regs unchanged.
```

(II) 4TH FIBONACCI NUMBER (FIB(0)=0, FIB(1)=1, …) 6

```
        MVI  B, 04H   ; n = 4
        MVI  C, 00H   ; F0
        MVI  D, 01H   ; F1
        MVI  E, 00H   ; counter
NEXT:
   MOV  A, C
        ADD  D        ; A = F(n-2) + F(n-1)
        MOV  C, D     ; shift
        MOV  D, A
        INR  E
        MOV  A, E
        CMP  B
        JNZ  NEXT
        MOV  A, D     ; A = Fib(4) = 03H
```

```
        STA  9000H
        HLT
```

Q6 Trace memory-fill program; explain DAA

7 + 3

(a) Trace LXI/SUB/MVI/MOV M,A/INX H/DCR D/JNZ — final A, D, HL

7

```
      LXI H, 8000H   ; HL = 8000H, pointer
      SUB B          ; A = A - B; assume A initially = B → A = 00H
      MVI D, 0FH     ; D = 15  (loop count)
LOOP:
 MOV M, A        ; (HL) = 00H
      INX H          ; HL ++
      DCR D          ; D --
      JNZ Loop       ; until D = 0
      HLT
```

The program clears 15 consecutive locations starting at 8000H. After execution:

- $A = 00$H (unchanged inside loop)
- $D = 00$H (decremented to zero)
- $HL = 800$FH (incremented 15 times from 8000H)

(b) Function of DAA

3

DAA (Decimal Adjust Accumulator) corrects the result of a binary addition between two BCD numbers so that A again contains a valid two-digit BCD number. Rule:

1. If the lower nibble of A > 9 *or* AC = 1, add 06H to A.
2. If the upper nibble of A > 9 *or* CY = 1, add 60H to A.

Example: 27 + 35 = 5CH (binary). After DAA → 62H = correct BCD (lower nibble C>9 ⇒ add 06).

Q7 RIM for pending interrupts; masking concept

7 + 3

(a) Use of RIM to sense pending interrupts

7

RIM (Read Interrupt Mask) loads A with status information:

BIT	MEANING
D_0	Mask state of RST 5.5 (1=masked)
D_1	Mask state of RST 6.5
D_2	Mask state of RST 7.5
D_3	IE flag (1 = interrupts enabled)
D_4	Pending RST 5.5
D_5	Pending RST 6.5
D_6	Pending RST 7.5
D_7	SID (serial input data)

Sample code to test for a pending RST 7.5:

```
RIM
ANI 40H   ; mask all but D6 (P7.5)
JNZ SERVICE_75
```

(B) MASKING — WHAT AND WHY 3

Masking is the act of selectively disabling individual interrupts so that the CPU ignores them even when they occur. It is required to (i) protect critical code sections from being preempted, (ii) prioritise certain devices over others by disabling lower-priority requests during high-priority service, and (iii) avoid spurious interrupts during initialisation.

Q8 SIM to mask vector interrupts — example 10

SIM WORD FORMAT AND EXAMPLE 10

SIM transfers the contents of A into the interrupt-mask register. The bit map is:

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
SOD	SOE	0	R7.5	MSE	M7.5	M6.5	M5.5

- **MSE** (D_3) = 1 enables the lower 3 mask bits to take effect.
- **M7.5/M6.5/M5.5**: 1 = mask (disable), 0 = unmask (enable).
- **R7.5** (D_4): writing 1 resets the RST 7.5 internal flip-flop.
- **SOE** (D_6) = 1 enables the SOD pin to receive the value of D_7.

Example — mask RST 5.5 only, leave RST 6.5 and RST 7.5 unmasked:

```
EI            ; global enable
MVI A, 09H    ; 0000 1001 → MSE=1, M5.5=1, M6.5=0, M7.5=0
SIM           ; write mask
```

The TRAP interrupt is non-maskable; SIM has no effect on it. INTR is enabled/disabled only by EI/DI.

— END OF 2023 SOLUTIONS —